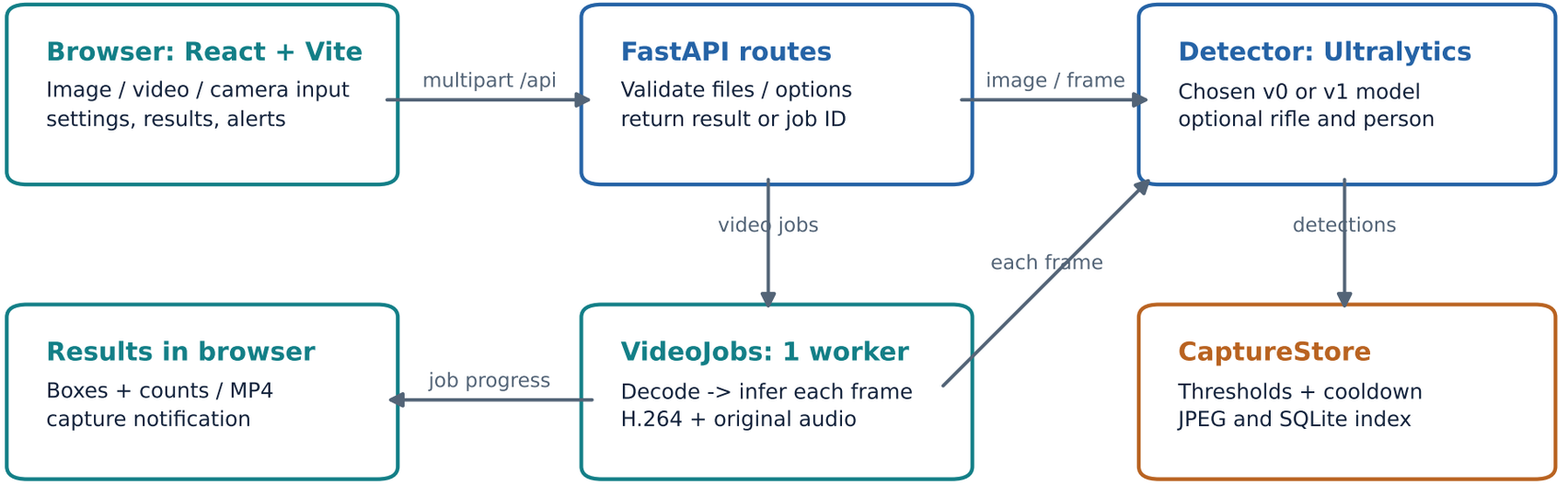
How the whole application works

High-level architecture - the models run locally; browser and API talk over HTTP



Where the data lives

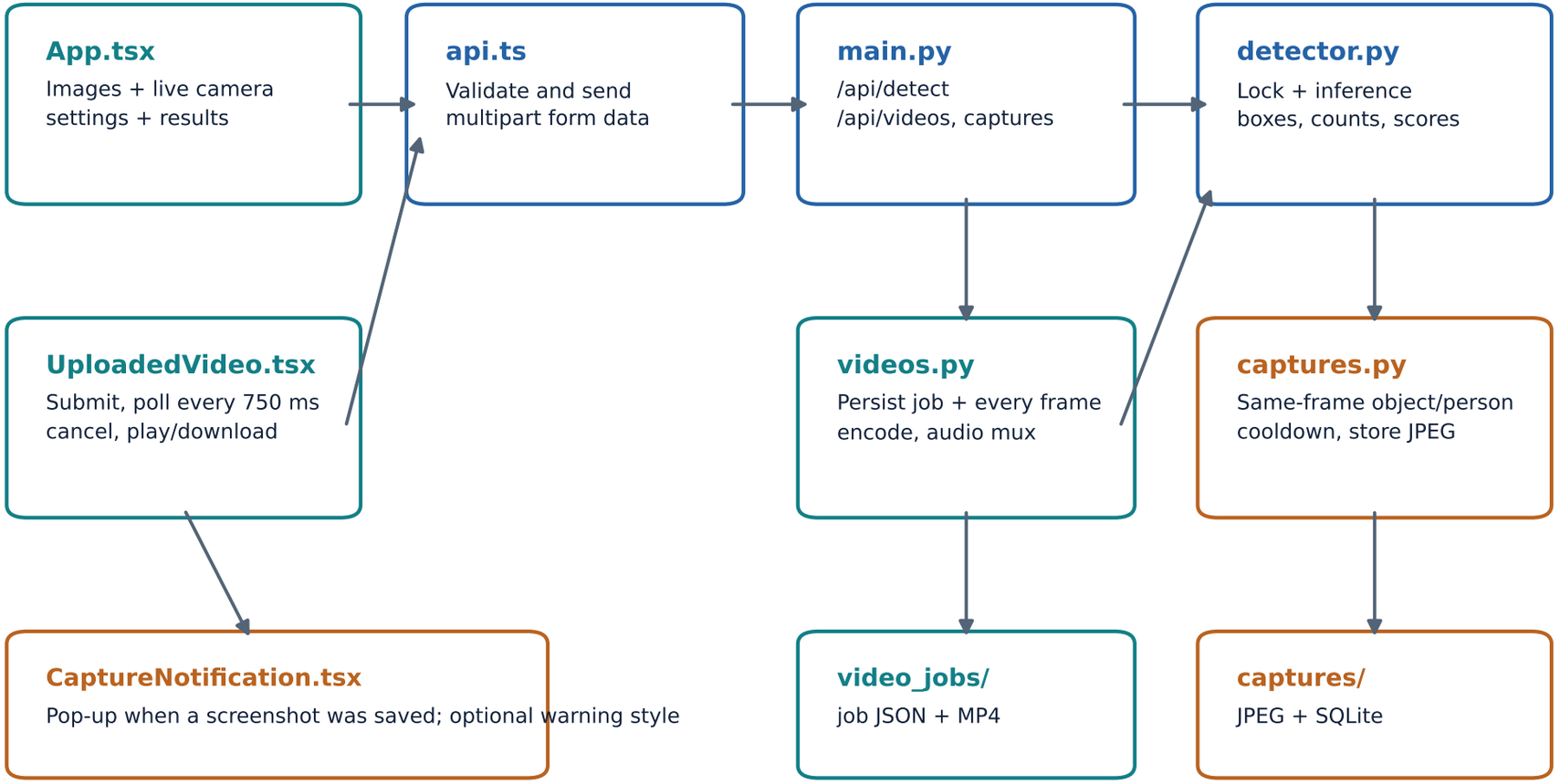
backend/models/: best.pt (v0), last.pt (v1), rifle_detector.pt, yolo11s.pt.

captures/: annotated JPEG files plus captures.sqlite3; video_jobs/: persisted job JSON plus output MP4.

No model training occurs in this running app; checkpoints were supplied or downloaded separately.

Components and connections

Detailed relationship diagram - routes, inference, job management and notifications



Three user journeys

Same detector and capture policy; different input and output lifecycles

1 IMAGE

1. User uploads, drops or pastes JPEG / PNG / WebP (up to 10 MiB).
2. Browser checks format/size and posts one file plus settings to /api/detect.
3. Backend decodes and runs selected v0/v1, plus rifle/person if selected.
4. Results: boxes, confidence, counts, weapon count, annotated JPEG.
5. Eligible screenshot -> JPEG + SQLite; response carries capture; UI shows notification.

2 RECORDED VIDEO

1. User uploads MP4 / WebM / MOV (up to 100 MiB) and starts detection.
2. POST /api/videos reserves a job; one background worker decodes EVERY frame.
3. Frames run through detector; timestamps retained; H.264 encoded, audio muxed.
4. Job status, count and latest capture saved to JSON; browser polls /api/videos/id.
5. Eligible frames save JPEGs; CaptureNotification shows during processing.
6. Completed MP4 becomes playable/downloadable; no red warning banner in frames.

3 LIVE CAMERA

1. Browser asks camera permission; video only, no microphone.
2. Canvas grabs one frame and posts it as an image to /api/detect.
3. Wait for response before grabbing another: no parallel frame backlog.
4. Result updates annotated preview, counts and approximate request FPS.
5. Eligible frames save screenshots and notify; Stop releases camera tracks.

Default capture: pistol / knife / rifle >= 35% AND person >= 55% in the same frame; 2 s cooldown.

Detection sliders filter candidates first (25% each by default); auto-capture can be disabled.

Models and recorded validation metrics

Read directly from local checkpoint metadata - aggregate box metrics, not live CCTV accuracy

Checkpoint / purpose	Classes	Precision	Recall	mAP@50	mAP@50:95
best.pt (v0)	6 objects	87.86%	82.27%	90.20%	74.66%
last.pt (v1; site default)	6 objects	81.17%	83.23%	87.84%	68.11%
rifle_detector.pt	rifle only	88.57%	87.70%	91.40%	64.64%
yolo11s.pt (COCO)	80 incl. person	70.58%	57.53%	63.83%	46.92%

Local CPU timing (Apple arm64, synthetic gray 640 x 480 input, 640px inference)

5 warmed predictions per model, median wall-clock: v0 59.8 ms | v1 57.5 ms | rifle 33.2 ms | person 58.6 ms

These timings EXCLUDE decode, upload, annotation, video encoding, disk, and UI. Real CCTV FPS is not measured.

Checkpoint numbers are stored validation summaries; validation images/labels and class-wise AP are absent here.

The person checkpoint numbers are for ALL 80 COCO classes, NOT person-class AP.

v0's recorded mAP is higher than v1's, but dataset splits may differ; neither value proves deployment accuracy.

Training provenance and honest interpretation

What the checkpoints prove, what they do not prove, and how to measure field performance

What we can verify locally

best.pt and last.pt contain six SoHAS classes: pistol, smartphone, knife, purse, banknote, card.

rifle_detector.pt is one-class rifle; yolo11s.pt is the 80-class COCO model, used only for person.

Serialized training settings record requested epochs: v0 50, v1 15, rifle 60, COCO 600; image size 640.

Training dataset references end in data.yaml for custom models; dataset content and labels were not found.

The application only INFERs with those weights; no training script, dataset split, or reproducible validation run is present.

How to obtain a meaningful AP for your CCTV footage

1. Prepare representative, consented CCTV images with ground-truth bounding boxes for pistol, knife and rifle.

2. Keep a held-out test split distinct from training; validate each checkpoint against the SAME images and labels.

3. Report per-class AP@50 and AP@50:95, precision/recall, false alerts, and small/distant-object results.

4. Benchmark end-to-end video throughput and camera latency on the actual deployment computer.

Sources: backend/detector.py, main.py, videos.py, captures.py; src/App.tsx, UploadedVideo.tsx, api.ts, types.ts;
local backend/models/*.pt metadata (read-only pickle opcode inspection); measured CPU benchmark, 5 warm runs.

Confidence is a model score, not a probability that someone is dangerous. Human verification is required.